N056 원가를 계산하고 크레딧 결제 붙이기_학습서

Chapter01

원가를 계산하고 크레딧 결제 붙이기

AI 친구 에이전트 서비스에서 사용자가 메시지를 하나 보낼 때마다 LLM API 비용이 나갑니다. 사용자가 늘수록 비용도 늘고, 지금은 그 비용을 아무도 내지 않습니다. 메시지 한 건에 실제로 얼마가 드는지, 무료로 얼마까지 줄 수 있는지, 넘어선 사용자에게는 얼마를 받아야 하는지, 그리고 돈을 받는 흐름은 어떻게 만들어야 안전한지를 오늘 차례로 정합니다. 계산은 내 서비스의 Langfuse 기록으로 하고, 결제는 토스페이먼츠 테스트 환경에서 실제 결제 흐름과 똑같이 만들어 봅니다.

주관식 퀴즈

Q. [시작 질문] 에이전트 서비스를 무료로 열어 두었더니 사용자 한 명이 하루에 메시지 300개를 보냅니다. 이 사용자 때문에 생기는 문제와, 막으려면 무엇을 정해야 하는지 떠오르는 대로 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. Langfuse 기록으로 메시지 한 건의 원가를 재고, 사용자 유형별 한 달 원가를 계산할 수 있습니다.
2. 같은 메시지로 두 모델의 원가와 답을 비교하고, 원가, 결제 수수료, 고정비로 크레딧 묶음 가격과 무료 한도를 정할 수 있습니다.
3. 크레딧을 잔액 하나가 아니라 원장(거래 기록)으로 설계하고, 서버에서만 차감되게 구현할 수 있습니다.
4. 결제 요청, 금액 검사, 서버 승인, 크레딧 지급으로 이어지는 결제 흐름을 시퀀스 다이어그램으로 설명하고 토스페이먼츠 테스트 결제로 구현할 수 있습니다.
5. 같은 결제가 두 번 승인되거나 금액이 바뀐 요청이 들어와도 크레딧이 잘못 지급되지 않는 것을 시험으로 확인할 수 있습니다.

Chapter02

시작 상태 맞추기

이 교안의 실습은 아래 상태가 갖춰져 있어야 진행할 수 있습니다. 크레딧과 결제는 내 컴퓨터에서 먼저 만들고 시험하므로, 저장소를 내 컴퓨터에서 실행할 수 있고 그 실행이 Supabase 시험용 프로젝트를 쓰면 됩니다. Langfuse 기록이 아직 없다면 이 섹션의 "Langfuse 기록이 없을 때"의 방법으로 원가를 추정하고 넘어갈 수 있습니다.

앞 실습을 이어서 하고 있다면 아래 확인 명령만 실행하고 넘어갑니다. 빠진 상태가 있으면 템플릿의 실습별 시작 상태 문서에서 "없으면" 칸의 항목을 표의 위 칸부터 차례로 실행합니다. 처음부터 모두 맞추면 한두 시간이 걸리므로, 빠진 항목이 많다면 수업 전에 맞춰 두거나 강사에게 먼저 알립니다.

필요한 상태	확인할 것	없으면
템플릿으로 만든 내 저장소, 가상 환경, 내 컴퓨터에서 실행	`uvicorn app.main:app --reload` 로 화면이 열림	A의 3번, 저장소 README의 로컬 실행 준비
`.env` 의 Supabase 연결(저장소와 익명 로그인)	화면 왼쪽 아래에 "게스트로 사용 중"	`docs/supabase.md` 1단계, `.env` 에 다섯 값
실제 LLM 키(`LLM_PROVIDER` 가 `fake` 가 아님)	답이 "(가짜 응답)"으로 시작하지 않음	`.env` 의 `LLM_PROVIDER`, `LLM_API_KEY`
Langfuse 계측(트레이스에 비용이 보임)	Langfuse 트레이스 목록에 비용 칸이 채워짐	없으면 이 섹션의 추정 방법으로 대신함

직접 실행할 명령: 시작 상태 확인

```
python -c "import httpx"
grep -c "^STORAGE=supabase" .env
grep -c "^LLM_PROVIDER=fake" .env
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
docs/start-states.md 의 표를 보고, 이 저장소와 지금 gcloud 프로젝트가 '원가와 결제' 실습을 시작할 상태인지 확인해 줘.
확인 명령만 실행하고 아무것도 만들거나 바꾸지 마. 빠진 항목이 있으면 문서의 어느 항목(A~I)을 실행해야 하는지 알려 줘.
```

메시지 한 건의 원가

원가(cost)는 서비스를 제공하는 데 드는 돈이고, 사용량에 따라 늘어나는 변동비(variable cost)와 사용량과 상관없이 매달 나가는 고정비(fixed cost)로 나눠 봅니다. 에이전트 서비스의 변동비는 대부분 LLM API 비용이고, 서버, 데이터베이스, 관측 도구, 결제 대행 계약에 드는 돈은 지금 규모에서는 고정비에 가깝습니다. 이 섹션에서는 변동비인 메시지 한 건의 원가를 재고, 고정비는 섹션 3에서 가격을 정할 때 더합니다.

메시지 한 건을 처리할 때 에이전트는 LLM을 한 번만 부르지 않습니다. 어떤 도구를 쓸지 고를 때, 캐릭터 말투로 답을 만들 때, 기억할 것을 고를 때 각각 부릅니다. LLM 제공사는 호출마다 입력 토큰과 출력 토큰의 수에 단가를 곱해 요금을 매기므로, 메시지 한 건의 원가는 이 호출들의 요금을 더한 값입니다.

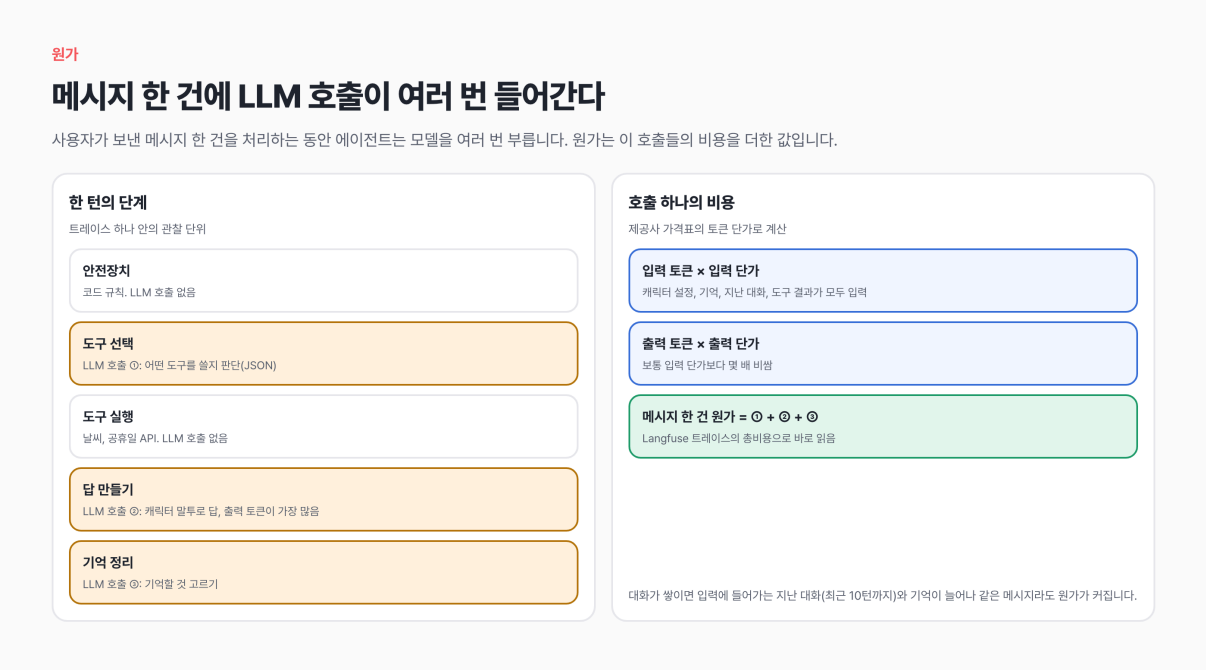


그림: 메시지 한 건을 처리하는 단계와 LLM 호출, 호출 하나의 비용 계산

그림에서 눈여겨볼 점은 입력 토큰입니다. 답을 만드는 호출에는 사용자의 메시지만 들어가는 것이 아니라 캐릭터 설정, 기억, 지난 대화, 도구 결과가 함께 들어갑니다. 그래서 같은 "안녕"이라도 대화가 쌓인 사용자의 메시지가 더 비쌉니다. 템플릿은 지난 대화를 최근 10턴까지만 넣으므로, 원가는 대화 열 턴 남짓까지 늘다가 그 뒤로는 요약과 기억 메모가 길어지는 만큼만 조금씩 늘어납니다. 원가를 평균 하나로만 보면 이런 차이를 놓치므로, 평균과 함께 비싼 쪽(상위 10% 정도)의 값과 대화 첫 메시지와 열 번째 이후 메시지의 차이도 봅니다.

Langfuse에서 원가 읽기

Langfuse는 generation 관찰 단위에 기록된 모델 이름과 토큰 수로 비용을 계산합니다. 모델 이름이 Langfuse의 가격표에 있으면 비용이 바로 나오고, 없으면 비용 칸이 비어 있습니다. 비용이 비어 있다면 Langfuse 프로젝트 설정의 모델 정의(Models)에 내가 쓰는 모

델의 이름과 토큰 단가를 직접 추가합니다. 단가는 LLM 제공사의 가격 페이지에서 확인합니다. 가짜 LLM(`LLM_PROVIDER=fake`)으로 보낸 메시지는 토큰과 비용이 0이므로, 원가를 읽으려면 실제 LLM으로 한 대화 안에서 15턴 넘게 이어지는 대화를 하나 이상 보내 둡니다.

직접 확인할 곳: Langfuse에서 원가 표 채우기

1. Langfuse > Tracing > Traces 에서 기간을 최근 7일로 두고, 에이전트 서비스의 대화 트레이스만 보이게 거릅니다(트레이스 이름이나 environment로).
2. 트레이스 몇 개를 열어 도구 선택, 답 만들기, 기억 정리 generation마다 모델, 입력·출력 토큰, 비용이 찍혀 있는지 봅니다. 비어 있으면 위의 모델 정의를 먼저 추가합니다.
3. 트레이스 목록의 비용 칸으로 메시지 한 건의 평균 비용과 가장 비싼 쪽 몇 건의 비용을 읽어 아래 표에 적습니다.
4. 15턴 넘게 이어진 대화의 트레이스를 시간순으로 놓고, 첫 메시지와 열 번째 이후 메시지의 비용과 입력 토큰을 비교합니다. 세션 화면(Sessions)이 있으면 대화 하나를 열어 보고, 없으면 사용자로 거른 트레이스 목록을 씁니다. 화면 이름은 Langfuse 버전에 따라 조금 다를 수 있습니다.

항목	내 서비스의 값	읽은 곳
지금 쓰는 모델		generation의 모델 이름
메시지 한 건 평균 비용		트레이스 비용 평균
비싼 쪽 메시지 비용(상위 10%)		비용 순으로 정렬한 트레이스
가장 비싼 단계		generation별 비용
대화 첫 메시지 비용		긴 대화의 첫 트레이스
열 번째 이후 메시지 비용		긴 대화의 뒤쪽 트레이스

LLM 제공사 요금은 보통 달러로 매겨지므로 원 단위로 바꿔 둡니다. 환율은 계산한 날의 값을 함께 적습니다.

Langfuse 기록이 없을 때

아직 계측하지 않았다면 LLM 제공사의 사용량 화면으로 추정합니다. 실제 LLM으로 메시지를 정확히 20개 보낸 뒤, 제공사 콘솔의 사용량(Usage) 화면에서 그 시간대의 입력·출력 토큰 수와 요금을 읽고 20으로 나눕니다. 다른 사람이 같은 키를 쓰고 있으면 섞이므로, 혼자 쓰는 키로 조용한 시간대에 합니다. 이 방법으로는 단계별 비용과 대화 길이에 따른 차이는 알 수 없고, 평균 하나만 얻습니다.

사용자 유형별 한 달 원가

수업 중에 보낸 메시지만으로는 실제 사용자가 하루에 메시지를 몇 개 보내는지 알 수 없습니다. 실제 사용자가 생기기 전에는 사용자 유형을 나눠 하루 메시지 수를 가정하고, 사용자가 생기면 Langfuse의 사용자별 트레이스 수로 바꿔 넣습니다. AI 친구 서비스에는 하루에 몇 마디만 나누는 사용자와 몇 시간씩 대화하는 사용자가 함께 있고, 자동화 프로그램으로 메시지를 쉬지 않고 보내는 계정도 생길 수 있습니다.

사용자 유형	하루 메시지(가정)	메시지 한 건 원가	한 달 원가(원가 × 하루 메시지 × 30)
보통 사용자	20		
많이 쓰는 사용자	300		
자동화 악용 계정	3,000		

많이 쓰는 사용자와 악용 계정은 대화가 길게 이어지므로, 메시지 한 건 원가에는 평균보다 열 번째 이후 메시지의 비용을 넣는 편이 실제에 가깝습니다.

결과 해석: 싼 메시지와 비싼 사용자

작은 모델을 쓰고 있다면 메시지 한 건의 원가는 몇 원이 안 되는 값으로 나오고, 대개 답 만들기 호출이 가장 비쌉니다. 메시지 한 건이 1.5원이라면 보통 사용자의 한 달 원가는 900원이라 신경 쓸 일이 아닌 것처럼 보이지만, 하루 300개를 보내는 사용자는 한 달에 1만 3,500원, 악용 계정 하나는 13만 원 넘게 씁니다. 원가가 사용자 수가 아니라 사용량에 비례하기 때문에, 가격과 한도는 평균 사용자가 아니라 사용량의 분포를 보고 정합니다. 메시지 한 건 원가는 쓰는 모델에 따라 몇십 배까지 달라지므로, 표에 적은 값이 어느 모델에서 잰 값인지도 함께 남겨 둡니다.

📚 원가 계산 참고 자료

- Langfuse: Token and cost tracking: Langfuse가 토큰과 비용을 계산하는 방식과, 가격표에 없는 모델을 정의하는 방법이 있습니다(영어).

Chapter03

모델에 따라 달라지는 원가

섹션 2의 원가는 지금 쓰는 모델에서 잰 값입니다. 같은 제공사 안에서도 모델마다 토큰 단가가 크게 다르고, 답의 말투나 정확도가 아쉬워 더 큰 모델로 바꾸면 원가도 단가 차이만큼 늘어납니다. 아래는 OpenAI 가격표의 일부입니다(2026년 11월 기준, 100만 토큰당 달러).

모델	입력	출력
gpt-4o-mini (템플릿의 `openai` 기본값)	0.15	0.60
gpt-4.1	2.00	8.00
gpt-4o	2.50	10.00

gpt-4o의 단가는 gpt-4o-mini의 열여섯 배쯤입니다. 다른 제공사도 작은 모델과 큰 모델의 단가가 열 배 넘게 차이 나는 경우가 흔하므로, 내가 쓰는 제공사의 가격 페이지에서 지금 모델과 한 단계 큰 모델의 단가를 찾아 둡니다.

직접 확인할 곳: 같은 메시지로 두 모델 비교하기

1. 보낼 메시지 10개를 메모장에 적어 둡니다. 인사, 날씨나 공휴일 묻기, 고민 털어놓기, 약속 잡기처럼 서비스에 자주 오는 종류를 섞고, 앞 메시지를 이어 받는 메시지도 두세 개 넣습니다.
2. 지금 모델로 브라우저의 시크릿 창을 열어 새 게스트로 10개를 순서대로 보냅니다.
3. `.env` 의 `LLM_MODEL` 에 한 단계 큰 모델 이름을 적고 서버를 껐다가 다시 켭니다. `--reload` 는 코드 변경만 다시 읽고 `.env` 변경은 다시 읽지 않습니다.
4. 새 시크릿 창에서 새 게스트로 같은 10개를 같은 순서로 보냅니다. 큰 모델로 10개를 보내도 비용은 대개 몇십 원 안쪽입니다.
5. Langfuse 트레이스 목록을 두 게스트 사용자로 각각 걸러 비용 합계와 평균을 적고, 같은 메시지에 대한 두 답을 나란히 읽어 어느 쪽이 나은지 고릅니다.
6. 비교가 끝나면 `LLM_MODEL` 을 원래 값으로 되돌리고 서버를 다시 켭니다.

항목	지금 모델	큰 모델
모델 이름		
메시지 10개 비용 합계		
메시지 한 건 평균		
답이 더 나았던 메시지 수(10개 중)		
차이가 컸던 메시지의 종류		

결과 해석: 원가 차이와 품질 차이

비용은 대개 단가 차이와 비슷한 배수로 벌어집니다. 모델마다 답의 길이가 달라 출력 토큰이 달라지므로 정확히 같은 배수는 아닙니다. 답의 차이는 메시지 종류에 따라 다르게 보이는데, 가벼운 인사나 맞장구에서는 두 답을 가르기 어렵고, 고민 상담처럼 앞뒤 맥락을 이어야 하는 메시지에서 차이가 드러나는 경우가 많습니다. 차이가 드러난 메시지가 적다면 작은

모델을 그대로 쓰고, 많다면 모든 호출을 바꾸는 대신 아래 표의 첫 줄처럼 답 만들기 호출에만 큰 모델을 쓰는 방법을 검토합니다. 열 개를 눈으로 읽은 비교는 감을 잡는 정도이므로, 실제로 모델을 바꾸기 전에는 평가 데이터셋으로 실험해 확인합니다.

원가를 줄이는 다른 방법

모델을 고르는 것 말고도 원가를 줄이는 방법이 있습니다. 원가가 절반이 되면 같은 가격에 무료 한도를 두 배로 줄 수 있습니다.

방법	하는 일	대가
단계마다 모델 나눠 쓰기	도구 선택, 기억 정리처럼 형식이 정해진 일은 작은 모델에 맡기고, 답 만들기에만 큰 모델을 씀	작은 모델이 판단을 틀릴 수 있으므로 평가 데이터로 바꾸기 전후를 비교해야 함
프롬프트 캐싱	매번 똑같이 앞에 들어가는 지시문과 캐릭터 설정을 제공사가 캐시해 두면 그 부분의 입력 단가가 낮아짐	제공사마다 조건(최소 길이, 유지 시간)이 다르고, 앞부분이 매번 바뀌면 효과가 없음
출력 토큰 상한	답의 최대 길이를 정함	너무 짧으면 답이 중간에 끊김
지난 대화 줄이기	입력에 넣는 지난 대화를 최근 10턴보다 줄이고 나머지는 요약으로 넣음	조금 전에 한 이야기를 기억하지 못할 수 있음
기억 정리 빈도	매 메시지가 아니라 몇 메시지마다 한 번 정리	기억이 늦게 반영됨

어느 방법이든 답의 품질이 함께 바뀔 수 있습니다. 비용이 줄었다는 숫자만 보고 바꾸지 말고, 평가 데이터로 품질이 유지되는지 확인한 뒤에 바꿉니다.

차감 단위 정하기

크레딧을 쓰기로 했다면 먼저 무엇을 할 때 크레딧이 몇 개 빠지는지를 정합니다. 여기서 차감 단위는 대화 세션 하나가 아니라 사용자가 보낸 메시지 하나입니다. 세션마다 받으면 한 세션 안에서 메시지를 끝없이 보낼 수 있어서 원가와 맞지 않습니다.

방식	사용자 입장	운영자 입장
메시지당 고정(예: 1개)	메시지 하나에 얼마인지 바로 앎	메시지마다 원가가 달라도 평균으로 감수함
쓴 토큰만큼(원가에 비례)	보내기 전에는 얼마가 빠질지 모름	원가와 정확히 맞고, 답이 끝난 뒤 계산해 차감함
기본은 고정, 비싼 기능만 더	대부분 1개, 예외만 알면 됨	원가 차이가 큰 기능만 따로 받음

에이전트 서비스는 지난 대화를 최근 10턴까지만 넣기 때문에 긴 대화의 메시지도 원가가 일정 수준에서 멈추고, 그래서 메시지당 고정으로 받아도 원가에서 크게 벗어나지 않습니다. 토큰만큼 받으면 사용자가 길게 털어놓을수록 크레딧이 더 빠져서 짧게 말하게 되는데, 친구처럼 대화하는 서비스에서는 바라지 않는 행동입니다. 그래서 오늘은 메시지 하나에 크레딧 1개로 시작합니다. 일정표 사진을 읽는 기능(`/api/ocr`)처럼 이미지가 들어가 입력 토큰이 크게 느는 기능이나, 사용자가 큰 모델을 고를 수 있게 하는 경우에는 그 기능에만 크레딧을 더 받는 방식을 검토합니다. 섹션 2의 표에서 원가 차이가 몇 배 나는 기능이 있는지 보고 정합니다.

크레딧 가격을 정하는 재료

LLM 서비스는 월정액보다 쓴 만큼 내는 방식이 원가 구조와 잘 맞습니다. 원가가 사용량에 비례하기 때문입니다. 오늘은 크레딧(credit) 방식을 씁니다. 사용자는 크레딧 묶음을 미리 사 두고, 메시지 하나를 보낼 때마다 크레딧 1개가 줄어듭니다. 크레딧이 없으면 메시지를 보낼 수 없습니다.

크레딧 가격을 정할 때는 아래 네 가지를 봅니다.

1. 메시지 한 건의 원가. 평균보다 비싼 쪽에 가까운 값으로 잡아야 많이 쓰는 사용자에게서 손해가 나지 않습니다.
2. 결제 수수료. 토스페이먼츠의 일반 등급 카드 결제 수수료는 결제 금액의 3.4%이고 부가세 10%가 별도라 3.74%가 됩니다(2026년 11월 공시 요금 기준, 매출 규모와 계약 조건에 따라 다름).
3. 고정비. 사용자가 늘어 무료 범위를 넘으면 Supabase Pro 플랜은 월 25달러부터, Langfuse Core 플랜은 월 29달러부터 나갑니다. 토스페이먼츠에도 가입비 22만 원과 연관리비 11만 원이 있습니다. 고정비는 메시지 수와 상관없이 나가므로 사용자가 적을수록 한 사람이 메워야 하는 몫이 커집니다.
4. 마진. 무료 사용자의 원가와 고정비를 메우고 남길 몫입니다.

원가가 작을 때의 가격

메시지 한 건 원가가 1~2원이면 수수료를 더해도 크레딧 1개는 몇 원이면 원가를 넘습니다. 그런데 몇 원짜리를 한 번씩 결제하게 할 수는 없습니다. 결제할 때마다 사용자가 카드 인증을 거쳐야 하고, 계좌이체처럼 건당 최저 수수료가 있는 결제 수단도 있기 때문입니다(토스페이먼츠 계좌이체는 2.0%, 건당 최저 200원). 그래서 크레딧은 1,000원에 100개처럼 묶음으로 팝니다.

원가가 정하는 것은 가격이 내려갈 수 있는 바닥까지입니다. 크레딧 100개를 1,000원에 팔든 3,000원에 팔든 원가는 넉넉히 메우므로, 실제 가격은 사용자가 그 대화에 얼마를 낼 마

음이 있는지와 비슷한 서비스가 얼마를 받는지로 정합니다. 원가가 손익을 크게 움직이는 곳은 무료 한도입니다. 무료 사용자의 원가는 전부 운영자가 내고 무료 사용자는 보통 유료 사용자보다 훨씬 많아서, 무료로 하루 몇 개를 주느냐가 한 달 손익을 가장 크게 바꿉니다. 무료 한도는 계정마다 걸리므로, 게스트 계정을 계속 새로 만들면 무료 크레딧을 반복해서 받을 수 있다는 점도 기억해 둡니다.

무료 한도는 처음 온 사용자가 서비스를 충분히 경험할 만큼은 주되, 한도를 넘는 사용자가 생길 만큼만 줍니다. 무료 한도 안에서 다 해결되면 아무도 크레딧을 사지 않고, 무료 사용자의 원가는 전부 운영자가 냅니다.

결제 단위 정하기

크레딧을 묶음으로 판다면 결제 한 번에 얼마를 받고 크레딧을 몇 개 줄지를 상품표(price table)로 정해 둡니다. 상품표는 상품 ID, 금액, 지급할 크레딧 수를 한 줄씩 적은 표이고, 섹션 6에서 서버 코드에 그대로 넣습니다. 아래는 크레딧 1개를 10원 안팎으로 잡은 예입니다.

상품 ID	금액	크레딧	크레딧 1개 가격
`credits_100`	1,000원	100	10원
`credits_550`	5,000원	550	약 9.1원
`credits_1200`	10,000원	1,200	약 8.3원

가장 작은 묶음은 보통 사용자가 며칠 쓸 만큼으로 잡습니다. 하루 20개를 보내는 사용자에게 100개는 닷새 분량이라, 너무 작으면 결제를 자주 해야 하고 너무 크면 처음 사기가 부담스럽습니다. 큰 묶음에 보너스 크레딧을 얹으면 미리 받는 돈이 늘지만 크레딧 1개 가격이 내려가므로, 가장 큰 묶음의 크레딧 1개 가격이 비싼 쪽 메시지 원가와 수수료를 넘는지 확인합니다. 큰 모델을 쓰는 메시지에도 크레딧 1개만 받는다면 이 확인이 가장 먼저 깨집니다. 묶음은 처음에는 두세 개로 시작하는 편이 화면과 결제 흐름을 시험하기 쉽습니다.

결제할 때 화면은 상품 ID만 서버로 보내고, 금액과 크레딧 수는 서버가 상품표에서 찾습니다. 화면이 금액이나 크레딧 수를 보내게 만들면 사용자가 요청을 고쳐 100원에 크레딧 1,200개를 사는 주문을 만들 수 있습니다. 사지 않은 크레딧을 환불할지, 크레딧에 유효기간을 둘지는 전자상거래 관련 법과 결제 대행사 약관을 확인해야 하므로, 실제로 판매하기 전에 따로 정합니다.

움직여 보는 원가와 가격

아래 계산기에 섹션 2에서 읽은 메시지 한 건 원가를 넣고, 사용자 수, 하루 메시지 수, 무료 한도, 유료 전환율(무료 사용자 가운데 크레딧을 사는 사람의 비율), 크레딧 가격, 월 고정비를 바꿔 봅니다. 크레딧 1개로는 남는데 전체로는 손해가 나는 경우가 있는지, 크레딧 가격을 올리는 것과 무료 한도를 줄이는 것 가운데 어느 쪽이 손익을 더 많이 바꾸는지 봅니다.

계산기의 기본값에서는 크레딧 1개를 팔 때마다 8원 넘게 남지만, 사용자 1,000명 전체로는 한 달에 40만 원 넘게 손해가 납니다. 유료 사용자 50명이 남기는 돈이 10만 원이 조금 안 되는데, 무료 사용자 950명의 원가만 42만 원이 넘고 여기에 고정비가 더해지기 때문입니다. 크레딧 가격을 두 배로 올리면 유료 사용자 50명의 몫만 늘어 손해가 30만 원 가까이 남고, 무료 한도를 하루 10개에서 3개로 줄이면 무료 사용자의 원가가 줄고 유료 사용자가 사는 크레딧이 늘어 손익이 거의 맞춰집니다. 다만 무료 한도를 줄이면 서비스를 충분히 써 보기 전에 떠나는 사람이 늘어 전환율이 떨어질 수 있고, 전환율은 운영자가 정할 수 없고 실제로 재 봐야 아는 값입니다. 그래서 처음에는 무료 한도를 작게 잡고, 전환율을 잰 뒤에 늘려 갑니다. 정한 값은 아래 표에 적어 두고 다음 섹션의 구현에 씁니다.

정할 것	내 서비스의 값	근거
차감 단위(메시지 1건 = 크레딧 몇 개)		
크레딧을 더 받는 기능(있다면)		섹션 2의 원가 차이
상품표(상품 ID, 금액, 크레딧)		원가 바닥, 비슷한 서비스의 가격
처음 받는 무료 크레딧		
하루 무료 메시지(있다면)		

객관식 퀴즈

Q. 메시지 한 건 원가가 1.5원인 서비스에서 크레딧 100개를 1,000원에 팔고, 무료로 하루 10개를 줍니다. 사용자 1,000명 가운데 5%만 크레딧을 사고, 한 달 손익이 적자입니다. 적자의 가장 큰 원인은 무엇인가요?

• 1. 무료 사용자 950명의 원가와 고정비를 유료 사용자 50명이 남기는 돈으로 메우지 못한다
• 2. 크레딧 1개 가격이 메시지 원가보다 낮아서 팔수록 손해다
• 3. 결제 수수료 3.74%가 크레딧에서 남는 돈을 대부분 가져간다
• 4. 원가가 사용자 수에 비례하므로 사용자가 1,000명이면 적자가 날 수밖에 없다

- •

  1. 무료 사용자 950명의 원가와 고정비를 유료 사용자 50명이 남기는 돈으로 메우지 못한다

- •

  2. 크레딧 1개 가격이 메시지 원가보다 낮아서 팔수록 손해다

- •

  3. 결제 수수료 3.74%가 크레딧에서 남는 돈을 대부분 가져간다

- •

4. 원가가 사용자 수에 비례하므로 사용자가 1,000명이면 적자가 날 수밖에 없다

Chapter04

원장으로 설계하기

크레딧을 사용자 테이블의 잔액 칸 하나로 두고 메시지마다 1씩 빼는 방법이 가장 쉬워 보입니다. 하지만 잔액이 이상해졌을 때 왜 그렇게 됐는지 알 수 없고, 두 요청이 같은 잔액을 동시에 읽고 고치면 한쪽의 변경이 사라집니다. 돈과 관련된 숫자는 보통 원장(ledger)으로 관리합니다. 원장은 크레딧이 늘고 준 기록을 한 줄씩 쌓기만 하는 테이블이고, 잔액은 그 사용자의 기록을 모두 더한 값입니다.

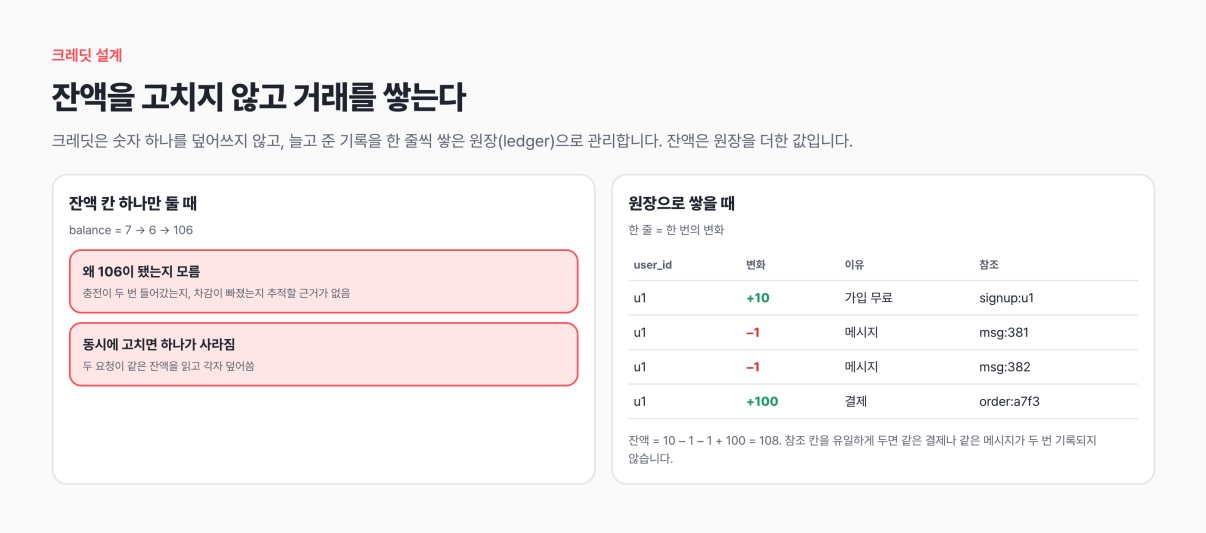


그림: 잔액 칸 하나로 관리할 때의 문제와 원장으로 쌓을 때의 모습

원장의 각 줄에는 누구의 변화인지, 얼마나 늘거나 줄었는지, 왜 바뀌었는지, 그리고 무엇 때문인지를 가리키는 참조를 적습니다. 참조 칸을 유일하게 두면 같은 결제로 크레딧이 두 번 들어가거나 같은 메시지로 두 번 빠지는 일을 데이터베이스가 막아 줍니다. 이 성질은 섹션 7의 중복 결제 막기에서 그대로 씁니다. 기록을 고치거나 지우지 않고 쌓기만 하므로, 잘못 차감했다면 그 줄을 지우는 대신 되돌리는 줄(+1)을 새로 쌓습니다.

구현하기 전에 정할 것

코딩 에이전트는 정해 준 구조대로 코드를 쓰고, 아래 질문의 답을 대신 정해 주지 않습니다. 표를 먼저 채운 뒤 요청합니다.

질문	고를 수 있는 것	내가 정한 것
언제 차감하나	메시지를 받자마자 / 답을 다 만든 뒤	
답을 만들다 실패하면	차감하지 않음 / 차감한 뒤 되돌리는 줄을 쌓음	
크레딧이 0이면	상태 코드는 402(Payment Required). 사용자에게 보여 줄 문구는 직접 정함	402, 문구:
처음 온 사용자	가입할 때 무료 크레딧 몇 개를 원장에 쌓나	
어디서 차감하나	서버(`/api/chat`)에서만. 화면은 잔액을 보여 주기만 함	서버
잔액을 보는 API	`GET /api/credits` 처럼 잔액과 최근 기록을 돌려줌	

차감은 반드시 서버에서 합니다. 화면 코드에서 크레딧을 확인하거나 빼면, 사용자가 브라우저 개발자 도구로 그 코드를 건너뛰고 `/api/chat` 을 바로 부를 수 있습니다. 크레딧 부족 응답에는 결제가 필요하다는 뜻의 표준 상태 코드 402를 씁니다. 401(로그인 필요)이나 500(서버 고장)을 쓰면 화면이 로그인을 다시 시키거나 모니터링이 장애로 잡고, 200에 오류 문구를 담으면 화면이 거절된 줄 모릅니다. 본문의 `error` 값은 화면이 읽어 크레딧 안내를 띄울 때 씁니다. 메시지를 받자마자 차감하면 동시에 여러 요청을 보내 잔액보다 많이 쓰는 것을 막기 쉽고, 답을 다 만든 뒤 차감하면 실패한 요청에 돈을 받지 않기 쉽습니다. 어느 쪽이든 고른 이유를 표에 적어 둡니다.

요청 틀(빈칸은 직접 채운다):

```
크레딧 원장을 추가해 줘. 앱 구조와 기존 테이블은 바꾸지 마.
- 테이블: supabase/migrations/0002_credits.sql 에 credit_ledger (user_id, 변화량 정수, 이유, 참조 문자열, 만든 시각). 참조는 유일하게.
  RLS 를 켜고 공개 정책은 만들지 마(서버의 secret key 로만 읽고 씀).
- 저장소: app/store 의 메모리 저장소와 Supabase 저장소에 같은 이름의 함수로 잔액 조회, 기록 추가를 만들어 줘.
- 차감 시점: [메시지를 받자마자 / 답을 다 만든 뒤], 실패하면 [차감 안 함 / 되돌리는 줄]
- 크레딧이 0이면 /api/chat 이 402 로 거절. 본문은 {"error": "insufficient_credits", "message": "[문구]", "balance": 0}
- 처음 온 사용자에게 무료 크레딧 [  ]개 (참조: signup:사용자ID)
- GET /api/credits: 잔액과 최근 기록 10줄
- 화면: static/index.html 에 남은 크레딧을 보여 주는 표시 하나. 차감 계산은 화면에 넣지 마.
- 테스트: 가짜 LLM으로 메시지를 보내면 잔액이 1 줄고, 잔액 0 이면 거절되는 테스트를 tests/ 에 추가.
바꾸기 전에 어떤 파일을 왜 바꾸는지 먼저 설명해 줘.
```

직접 확인할 곳: 크레딧 구현 확인

1. Supabase SQL Editor 에서 새 마이그레이션 파일의 내용을 실행하고, Table Editor 에 `credit_ledger` 가 생겼는지, RLS가 켜져 있는지 봅니다.
2. `pytest -q` 로 기존 테스트와 새 테스트가 모두 통과하는지 봅니다.

3. 내 컴퓨터에서 실행하고 새 브라우저(시크릿 창)로 열어, 화면에 처음 받은 무료 크레딧 수가 보이는지 봅니다.
4. 메시지를 하나 보내고 표시가 1 줄었는지, `credit_ledger` 에 −1 줄이 생겼는지 봅니다.
5. Table Editor 에서 내 사용자의 기록을 더해 잔액이 0이 되게 −줄을 직접 하나 넣은 뒤 메시지를 보내, 정한 문구로 거절되는지 봅니다.

▼ **힌트**

처음 온 사용자의 무료 크레딧은 `/api/me` 처럼 로그인한 사용자가 처음 부르는 API에서 "원장에 signup 참조가 없으면 쌓는다"로 처리하면 됩니다. 참조가 유일하므로 두 번 불러도 한 번만 쌓입니다. 잔액은 원장의 변화량 합계입니다. Supabase 저장소에서 합계를 구하는 방법은 PostgREST의 집계나 데이터베이스 함수(RPC)가 있는데, 에이전트에게 공식 문서 근거와 함께 고르게 합니다. 동시에 보낸 요청 두 개가 같은 잔액 1을 보고 둘 다 통과하는 문제가 신경 쓰인다면, 차감을 데이터베이스 함수 하나로 묶어 그 안에서 잔액 확인과 기록 추가를 함께 하는 방법을 찾아봅니다.

결과 해석: 잔액이 아니라 기록이 남는다

메시지를 보낼 때마다 원장에 한 줄씩 쌓이고, 화면의 숫자는 그 합계입니다. 잔액이 이상하면 원장을 시간순으로 읽어 어느 줄에서 틀어졌는지 찾을 수 있습니다. 거절은 서버에서 일어나므로, 화면의 표시를 지우거나 개발자 도구로 요청을 직접 보내도 크레딧 없이는 답을 받을 수 없습니다. 다만 지금은 크레딧이 줄기만 하고 늘릴 방법이 없습니다. 다음 섹션에서 결제로 크레딧을 늘리는 흐름을 설계합니다.

객관식 퀴즈

Q. 크레딧 잔액을 화면 코드(JavaScript)에서 확인하고, 0이면 보내기 버튼을 막도록 만들었습니다. 이 구현의 가장 큰 문제는 무엇인가요?

• 1. 버튼이 회색이 되어 보기 좋지 않다
• 2. 사용자가 화면 코드를 건너뛰고 서버 API를 직접 부르면 크레딧 없이도 메시지를 보낼 수 있다
• 3. 잔액 계산이 느려진다
• 4. 원장을 쓸 수 없게 된다**제출**

- •

  1. 버튼이 회색이 되어 보기 좋지 않다

- •

2. 사용자가 화면 코드를 건너뛰고 서버 API를 직접 부르면 크레딧 없이도 메시지를 보낼 수 있다

- •

  3. 잔액 계산이 느려진다

- •

  4. 원장을 쓸 수 없게 된다

🚀 더 해보기: 비싼 기능에 크레딧 더 받기

1. 일정표 사진 읽기(`/api/ocr`)에 크레딧을 더 받기

- 접근 힌트: Langfuse에서 사진 한 장을 읽는 호출의 비용을 메시지 한 건 원가와 비교해 몇 배인지 보고, 차감 개수를 정해 `/api/ocr` 에도 원장 차감을 넣어 달라고 요청한다. 참조는 메시지와 겹치지 않게 다른 접두사를 쓴다
- 성공하면: 원장에 이유가 다른 −줄이 쌓이고, 잔액이 모자라면 사진 읽기도 거절된다

2. 차감 개수를 상품표처럼 서버 설정으로 빼기

- 접근 힌트: 기능별 차감 개수를 코드 곳곳에 숫자로 적지 말고 한곳에 모아 두게 한다
- 성공하면: 모델이나 원가가 바뀌었을 때 한 줄만 고치면 된다

Chapter05

결제 대행사와 결제 승인

카드 결제를 직접 처리하려면 카드사마다 계약하고 카드 정보를 안전하게 다루는 보안 인증을 받아야 합니다. 그래서 대부분의 서비스는 결제 대행사(PG, payment gateway)를 씁니다. 오늘은 토스페이먼츠의 결제위젯을 씁니다. 결제위젯은 결제 수단을 고르고 카드사 인증을 받는 화면을 토스페이먼츠가 대신 띄워 주는 도구라서, 우리 서버는 카드 번호를 한 번도 보지 않습니다.

결제는 두 단계로 나뉩니다. 사용자가 결제창에서 카드사 인증을 마치는 것이 결제 인증이고, 그 결과를 받은 우리 서버가 토스페이먼츠 API에 "이 결제를 확정해 달라"고 요청하는 것이 결제 승인입니다. 승인 요청이 성공해야 비로소 돈이 빠져나가고 결제가 끝납니다. 인증만 하고 승인하지 않은 결제는 일정 시간이 지나면 사라집니다.

아래 시퀀스 다이어그램에서 다음을 누르면 요청이 한 단계씩 움직이고, 위쪽의 주문 상태, 카드 결제 여부, 크레딧 잔액이 함께 바뀝니다. 정상 결제를 끝까지 따라간 뒤, 상황을 "주소의 금액을 고친 경우"로 바꿔 서버가 어느 단계에서 멈추는지 비교합니다.

그림을 따라 읽기

1~3번에서 사용자가 크레딧 묶음을 고르면 서버는 결제를 시작하기 전에 주문을 데이터베이스에 저장합니다. 주문에는 주문 번호(orderId), 금액, 상태(대기)를 적습니다. 4번에서 브라우저는 결제위젯으로 결제를 요청하고, 카드사 인증이 끝나면 5번에서 토스페이먼츠가 브라우저를 우리가 정해 둔 성공 주소(successUrl)로 보냅니다. 이 주소의 쿼리에 결제 키(paymentKey), 주문 번호, 금액(amount)이 붙어 옵니다.

핵심은 6번입니다. 5번의 주소는 브라우저의 주소창에 그대로 보이므로, 사용자가 `amount=1000` 을 `amount=100` 으로 바꿔서 서버에 보낼 수도 있습니다. 그래서 서버는 받은 금액을 그대로 믿지 않고, 2번에서 저장해 둔 주문의 금액과 같은지, 주문이 아직 대기 상태인지 먼저 확인합니다. 둘 다 맞을 때만 7번에서 시크릿 키로 결제 승인 API를 부르고, 8번에서 승인이 성공하면 9번에서 주문을 완료로 바꾸고 원장에 크레딧을 쌓습니다.

결제에서 지킬 것

지킬 것	이유
주문을 먼저 서버에 저장	승인할 때 비교할 기준 금액이 있어야 함
successUrl의 금액을 저장한 금액과 비교	브라우저가 주소를 바꿀 수 있음
승인은 서버가 시크릿 키로	시크릿 키가 화면 코드에 있으면 누구나 그 키로 결제를 승인하거나 취소할 수 있음
크레딧은 승인 성공 뒤에만	successUrl로 이동했다는 것만으로는 돈이 들어온 것이 아님
같은 주문은 한 번만 처리	새로고침이나 네트워크 재시도로 승인 요청이 두 번 올 수 있음

토스페이먼츠에는 키가 두 종류 있습니다. 클라이언트 키는 결제위젯을 띄울 때 화면 코드에 넣는 키로, 드러나도 됩니다. 시크릿 키는 결제 승인처럼 돈이 움직이는 API를 부를 때 쓰는 키로, 서버에서만 쓰고 저장소와 화면 코드에 절대 넣지 않습니다. 내 컴퓨터에서는 `.env` 에, 배포한 서비스에서는 Secret Manager에 둡니다.

PG사 고르기

결제 대행사(PG, payment gateway)는 카드사와 간편결제 회사와의 연결을 대신 맡는 회사입니다. 오늘은 전자결제 계약 없이 연동 문서에 공개된 테스트 키로 전체 흐름을 시험할 수 있는 토스페이먼츠를 쓰지만, 실제 서비스에서는 상황에 따라 다른 곳을 고르기도 합니다.

상황	고르는 곳	대가
PG사를 여럿 쓰거나 나중에 바꿀 수 있어야 함	포트원 같은 결제 연동 플랫폼. 한 번 연동하면 콘솔 설정으로 PG사를 바꾸거나 결제 수단별로 나눔	PG사 계약은 따로 해야 하고 연동 계층이 하나 늘어남
해외 사용자에게 달러로 팔고 나라별 세금 처리를 맡기고 싶음	Paddle 같은 MoR(merchant of record). 대행사가 판매 주체가 되어 세금 신고와 영수증을 처리	건당 고정 수수료가 있어 작은 묶음에 불리함
휴대폰 앱 안에서 크레딧을 팖	스토어 결제(Google Play 결제, Apple 인앱 결제). 디지털 상품은 스토어 정책상 스토어 결제를 써야 함	수수료가 PG사보다 큼

어느 곳을 고르든 주문을 서버에 저장하고, 결제 뒤 서버가 금액을 확인하고, 승인이 확인된 뒤에만 한 번 크레딧을 지급하는 구조는 같습니다. 달라지는 것은 수수료 구조이고, 건당 고정 수수료가 있는 곳을 쓰면 섹션 3에서 정한 가장 작은 묶음을 키워야 수수료가 결제액에서 차지하는 몫이 줄어듭니다.

객관식 퀴즈

Q. 결제 성공 페이지(successUrl)가 열리면 화면 코드가 바로 `/api/credits/add` 를 불러 크레딧을 100개 늘리게 만들었습니다. 이 설계의 문제로 가장 알맞은 것은 무엇인가요?

• 1. 성공 페이지가 열렸다는 것은 카드 인증이 끝났다는 뜻일 뿐 결제가 승인된 것이 아니고, 누구나 그 API를 직접 불러 크레딧을 늘릴 수 있다
• 2. 크레딧 100개는 너무 많다
• 3. 성공 페이지는 서버가 아니라 토스페이먼츠가 보여 줘야 한다
• 4. 결제 수수료가 두 번 나간다**제출**

- •

  1. 성공 페이지가 열렸다는 것은 카드 인증이 끝났다는 뜻일 뿐 결제가 승인된 것이 아니고, 누구나 그 API를 직접 불러 크레딧을 늘릴 수 있다

- •

  2. 크레딧 100개는 너무 많다

- •

  3. 성공 페이지는 서버가 아니라 토스페이먼츠가 보여 줘야 한다

- •

  4. 결제 수수료가 두 번 나간다

📚 결제 흐름 참고 자료

- 토스페이먼츠: 결제위젯 연동하기: 결제위젯을 띄우고, 결제를 요청하고, successUrl의 값으로 서버가 승인하는 전체 흐름이 단계별로 있습니다.
- 토스페이먼츠: 멱등키: 같은 요청을 두 번 보냈을 때 한 번만 처리되게 하는 Idempotency-Key 헤더의 규칙이 있습니다.
- 토스페이먼츠: AI로 결제 연동하기: 코딩 에이전트에 토스페이먼츠 MCP, llms.txt, LLM Quick Reference를 연결하는 도구별 방법이 있습니다.
- NHN KCP 개발자센터: 국내 PG사 NHN KCP의 연동 가이드입니다. 거래등록, 결제승인, 거래취소로 이어지는 흐름이 토스페이먼츠와 같은 구조이고, MCP로 결제를 연동하는 안내도 있습니다.
- KG이니시스: INIpay PRO 일반결제: 국내 PG사 KG이니시스의 연동 매뉴얼입니다. 결제요청, 인증결과, 승인요청, 승인결과의 네 단계로 나뉘어 있어, 인증과 승인을 나누는 구조를 다른 PG사에서도 확인할 수 있습니다.

Chapter06

테스트 키 준비

결제위젯에 쓰는 내 상점의 연동 키(클라이언트 키는 `test_gck`, 시크릿 키는 `test_gsk` 로 시작)는 토스페이먼츠에 전자결제를 신청하고 승인을 받은 뒤에야 개발자센터에서 확인할 수 있습니다. 개발자센터에 가입만 해서는 결제위젯용 키를 받을 수 없으므로, 오늘은 결제위젯 연동 문서의 코드 예제에 공개된 문서용 테스트 키를 씁니다. 테스트 키로 한 결제는 승인이 가상으로 이루어져 실제로 돈이 빠져나가지 않습니다.

문서용 테스트 키는 모든 사람이 함께 쓰는 키라서 내 키와 두 가지가 다릅니다. 하나는 이 키로 한 결제가 개발자센터의 결제 내역과 로그에 남지 않는다는 점입니다. 그래서 오늘은 결제 결과를 개발자센터 대신 우리 서버가 받은 승인 응답과 `orders` 테이블로 확인합니다. 다른 하나는 시크릿 키까지 문서에 공개되어 있다는 점입니다. 지금은 지켜야 할 값이 아니지만, 전자결제 계약 뒤 내 키로 바꾸면 같은 자리의 값이 결제를 승인하고 취소하는 자격증명이 됩니다. 그래서 오늘도 시크릿 키를 실제 시크릿과 똑같이 `.env` 와 Secret Manager로만 다루고, 나중에 키를 바꿀 때 코드는 그대로 두고 값만 바꾸면 되게 만듭니다.

두 키를 `.env` 에 넣습니다. 클라이언트 키는 화면으로 보내도 되는 값이므로 서버가 화면에 넘겨주게 하고, 시크릿 키는 서버만 읽습니다.

직접 확인할 곳: 테스트 키 넣기

1. 토스페이먼츠 결제위젯 연동 문서의 코드 예제에서 문서용 테스트 키를 찾습니다. `test_gck` 로 시작하는 클라이언트 키와 `test_gsk` 로 시작하는 시크릿 키를 같은 문서에

서 한 세트로 가져옵니다. 세트가 아닌 두 키를 섞으면 INVALID_API_KEY 오류가 납니다.

2. 편집기로 저장소의 `.env` 를 열어 `TOSS_CLIENT_KEY`, `TOSS_SECRET_KEY` 두 줄을 추가하고 값을 직접 붙여 넣습니다.
3. `.env.example` 에는 같은 이름을 값 없이 추가해, 다른 사람도 어떤 값이 필요한지 알게 합니다.
4. `git status` 로 `.env` 가 커밋 대상에 없는지 봅니다.

토스페이먼츠 MCP 연결

결제위젯 스크립트 주소, 함수 이름, 승인 API의 인증 방식처럼 토스페이먼츠에 맞춰야 하는 세부는 버전마다 바뀌고, 코딩 에이전트가 기억하는 내용은 옛 버전(v1)과 섞여 있을 수 있습니다. 토스페이먼츠는 코딩 에이전트가 공식 문서를 직접 찾아 읽게 하는 MCP 서버(`tosspayments-integration-guide`)를 제공합니다. MCP를 연결하면 에이전트가 결제위젯 v2 문서를 기본으로 찾아 근거로 쓰므로, 교안에서는 이런 세부를 외우거나 요청에 적지 않습니다.

MCP가 알려 주는 것은 토스페이먼츠 쪽의 규칙뿐입니다. 상품표를 어디에 둘지, 언제 크레딧을 지급할지, 금액을 무엇과 비교할지는 우리 서비스의 구조라서 MCP 문서에 없고, 에이전트도 대신 정해 주지 않습니다. 그래서 아래 요청은 짧게 쓰고 토스페이먼츠 세부는 "토스페이먼츠 MCP로 찾아 근거로 써 줘"로 맡기며, 우리 서비스의 구조는 에이전트가 물어볼 때 답하고 만든 뒤 확인 목록으로 점검합니다.

직접 실행할 명령: 토스페이먼츠 MCP 추가 (Claude Code)

```
# Node.js 가 설치되어 있어야 합니다(npx 로 실행). 저장소 루트에서
claude mcp add tosspayments-integration-guide -- npx -y @tosspayments/integration-guid
e-mcp@latest
```

추가한 뒤 Claude Code를 다시 열고 `/mcp` 로 `tosspayments-integration-guide` 가 연결됨으로 보이는지 확인합니다. 다른 코딩 에이전트를 쓴다면 토스페이먼츠의 AI 연동 안내 페이지에 도구별 설정 방법이 있고, MCP를 쓸 수 없는 도구라면 `https://docs.tosspayments.com/llms.txt` 주소를 요청에 함께 붙여 넣습니다. 에이전트가 MCP를 알아서 쓰지 않을 때도 있으므로, 요청마다 "토스페이먼츠 MCP로"라고 적고 답에 어떤 문서를 근거로 썼는지 적게 합니다.

구현할 구조 정하기

결제는 화면, 서버 API 두 개, 데이터베이스 테이블 하나로 만듭니다. 화면은 지금의 웹 화면(`static/index.html`)에 크레딧 안내 팝업 하나와 결제위젯이 뜨는 영역, 성공 페이지와 실패 페이지 두 개만 더합니다.

구성 요소	하는 일
`GET /api/products`	상품표를 돌려줌. 화면은 이 목록으로 상품을 보여 주고, 상품표를 화면 코드에 따로 적지 않음
`orders` 테이블	주문 번호(유일), 사용자, 상품 ID, 금액, 지급할 크레딧, 상태(대기·완료·실패), paymentKey, 승인 시각, 만든 시각
`POST /api/orders`	화면이 보낸 상품 ID로 상품표에서 금액과 크레딧을 찾아 주문을 대기 상태로 저장하고 주문 번호와 금액을 돌려줌(화면이 보낸 금액을 쓰지 않음)
크레딧 안내 팝업	남은 크레딧, 차감 규칙, 상품 목록을 보여 주고 상품을 고르면 결제로 넘어감. 아래에서 자세히 정함
결제 화면	클라이언트 키로 결제위젯을 띄우고, 주문 번호와 금액으로 결제 요청. successUrl과 failUrl을 지정
성공 페이지	주소의 paymentKey, orderId, amount를 서버로 보냄. 크레딧을 직접 늘리지 않음
`POST /api/payments/confirm`	주문 금액과 상태 확인 → 토스페이먼츠 결제 승인 API 호출 → 성공하면 주문 완료와 원장 +주문의 크레딧 수(참조: 주문 번호)
실패 페이지	주소의 오류 코드와 문구를 보여 주고 주문을 실패로 바꿈

표의 각 칸이 하는 일은 우리가 정한 구조이고, 에이전트가 스펙을 물어 오면 이 표가 답의 근거가 됩니다. 결제위젯을 띄우는 코드, 승인 API의 주소와 인증 헤더처럼 토스페이먼츠에 맞춰야 하는 부분은 에이전트가 MCP로 찾아 채웁니다.

크레딧 안내 팝업 설계

결제 흐름이 아무리 안전해도, 사용자가 크레딧이 왜 줄었는지, 다 쓰면 어떻게 되는지, 어디서 사는지를 모르면 결제까지 오지 않습니다. 그래서 결제위젯을 띄우기 전에 크레딧을 안내하고 상품을 고르는 팝업(모달)을 하나 둡니다. 새 페이지를 만들지 않고 대화 화면 위에 띄우면, 크레딧이 떨어져 거절된 메시지를 입력창에 그대로 둔 채 충전하고 다시 보낼 수 있습니다.

언제 여나	화면이 하는 일
사용자가 남은 크레딧 표시나 크레딧 사기 버튼을 누를 때	팝업을 엶

언제 여나	화면이 하는 일
`/api/chat` 이 크레딧 부족(402, `error` 가 `insufficient_credits`)으로 거절할 때	거절 문구와 함께 팝업을 열고, 보내려던 메시지는 입력창에 남겨 둠
잔액이 적을 때(예: 10개 이하)	팝업을 바로 열지 않고 입력창 위에 "크레딧이 얼마 남지 않았어요" 띠와 사기 버튼을 보여 줌

팝업에는 다섯 가지를 보여 줍니다. 남은 크레딧, 차감 규칙(메시지 1개에 크레딧 1개, 더 받는 기능이 있다면 그 기능과 개수), `GET /api/products` 로 받은 상품 목록(금액, 크레딧 수, 크레딧 1개 가격), 결제하기 전에 무엇을 얼마에 사는지와 환불 기준을 적은 안내 문구, 그리고 게스트로 쓰는 중이라면 브라우저 데이터를 지우면 산 크레딧에 다시 접근할 수 없다는 경고입니다. 환불 기준 문구는 섹션 3의 결제 단위에서 미뤄 둔 정책이 정해지기 전까지 자리만 만들어 둡니다. 남은 크레딧과 상품 목록은 모두 서버가 준 값을 그대로 보여 주고, 화면에서 계산하거나 따로 적어 두지 않습니다.

결제 만들기

요청은 짧게 쓰고, 구조는 에이전트와 대화하면서 확정합니다. 에이전트가 상품표, 주문에 저장할 값, 팝업 문구, 승인 전에 검사할 것 같은 질문을 정리해 오면 위의 구조 표, 팝업 설계, 섹션 5의 시퀀스 다이어그램, 섹션 3에서 정한 상품표를 근거로 답합니다.

코딩 에이전트에게 맡길 때: 결제 기능 만들기

```
토스페이먼츠 결제위젯 v2 로 크레딧을 사는 흐름을 만들어 줘.
토스페이먼츠 쪽 코드는 토스페이먼츠 MCP로 결제위젯 v2 연동 문서를 찾아 근거로 쓰고, 근거로 쓴 문서를 알려 줘.
결제 기능을 붙이기 위해 필요한 추가 스펙은 나한테 물어봐서 확정된 이후에 만들어 줘.
```

확인 목록

에이전트가 모든 것을 묻지는 않으므로, 질문 목록을 받으면 아래 항목이 다 들어 있는지 먼저 보고 빠진 항목은 직접 덧붙여 답합니다. 코드가 만들어진 뒤에도 같은 목록으로 하나씩 점검합니다. 에이전트에게 물어 확인하는 항목은 설명만 듣지 말고 해당 파일과 줄을 함께 받습니다.

확인할 것	어디서 보나
상품표가 서버 코드 한 곳에 있고 `GET /api/products` 로 내려옴	브라우저에서 `/api/products` 를 열어 섹션 3의 상품표와 같은지 봄
`POST /api/orders` 가 상품 ID만 받고, 금액과 크레딧 수는 서버가 상품표에서 찾음	에이전트에게 화면이 보낸 금액을 쓰는 코드가 있는지 묻기

확인할 것	어디서 보나
`orders` 테이블에 주문 번호(유일), 사용자, 상품 ID, 금액, 크레딧 수, 상태, paymentKey, approved_at 칸이 있고 RLS가 켜져 있음	Supabase SQL Editor에서 마이그레이션을 실행한 뒤 Table Editor
주문 번호가 추측할 수 없는 무작위 값	`orders` 에 쌓인 주문 번호 두세 개를 비교
시크릿 키는 `.env` 의 `TOSS_SECRET_KEY` 에서만 읽고, 화면과 `/api/config` 에 없음	브라우저에서 `/api/config` 를 열어 `test_gsk` 가 없는지 봄
크레딧을 늘리는 코드가 서버의 승인 처리 한 곳뿐이고, 성공 페이지는 받은 값을 서버로 보내기만 함	에이전트에게 크레딧을 늘리는 코드 위치를 모두 묻기
서버 로그에 시크릿 키와 paymentKey 전체가 없음	결제한 뒤 서버 터미널 로그에서 `test_gsk` 와 paymentKey 를 검색

직접 확인할 곳: 실행 확인

1. 내 컴퓨터에서 실행하고 크레딧 표시를 눌러 팝업에 남은 크레딧, 차감 규칙, 상품 목록이 보이는지 봅니다. 게스트로 연 화면이면 크레딧 보존 경고도 보여야 합니다.
2. 섹션 4에서처럼 잔액을 0으로 만든 뒤 메시지를 보내, 거절 문구와 함께 팝업이 열리고 보내려던 메시지가 입력창에 남아 있는지 봅니다.
3. 팝업에서 상품을 고르면 결제위젯이 뜨는지, `orders` 에 고른 상품 ID, 금액, 크레딧 수가 담긴 대기 상태 줄이 생기는지 봅니다.
4. 테스트 카드로 결제를 끝까지 진행해 성공 페이지에 충전된 크레딧과 잔액이 보이는지 봅니다.
5. `orders` 의 그 주문이 완료 상태이고 paymentKey와 approved_at 이 채워졌는지, `credit_ledger` 에 `order:주문번호` 참조로 + 줄이 생겼는지 봅니다. 문서용 테스트 키로 한 결제는 개발자센터 결제 내역에 남지 않으므로, approved_at 이 토스페이먼츠가 승인했다는 기록입니다.
6. 충전한 크레딧으로, 입력창에 남아 있던 메시지를 다시 보내 잔액이 줄어드는지 봅니다.

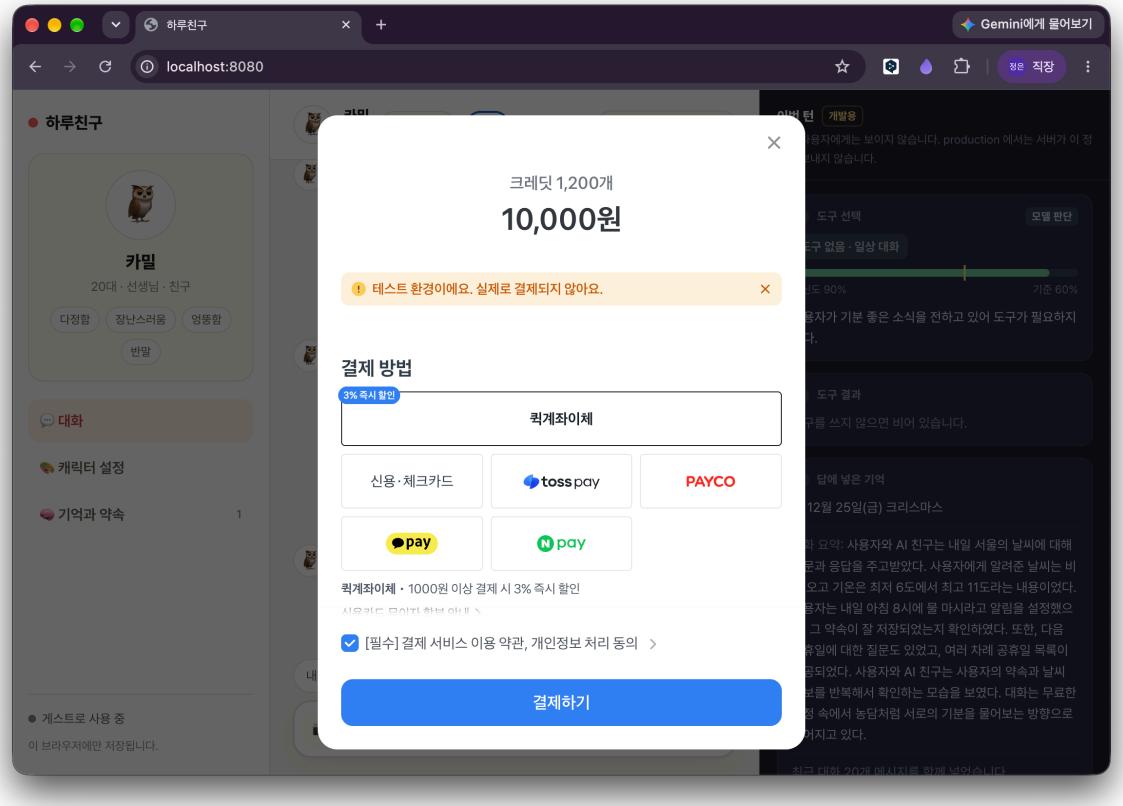


그림: 내 컴퓨터에서 연 에이전트 서비스 화면: 크레딧 안내 팝업에서 상품을 고른 뒤 토스페이먼츠 결제위젯이 뜬 모습

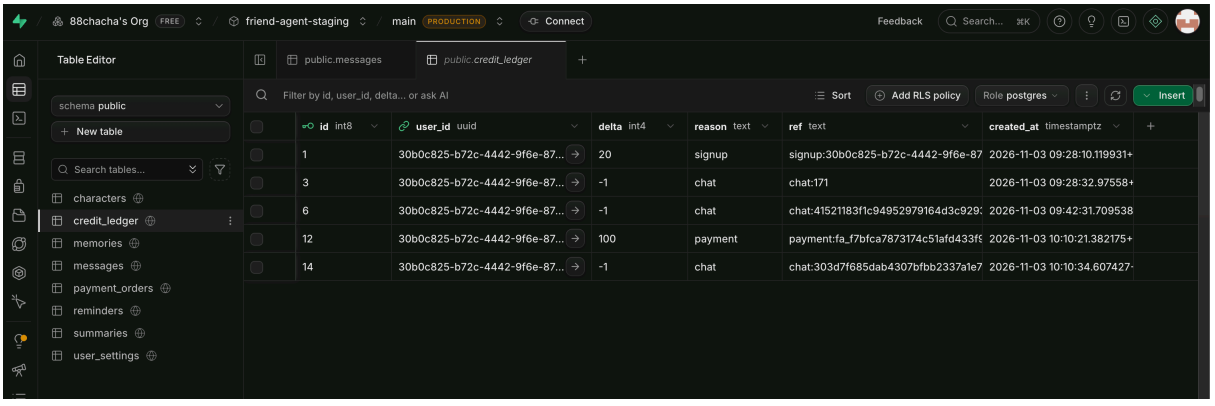


그림: Supabase 대시보드 > Table Editor > credit_ledger: 가입 무료 크레딧, 메시지 차감, 결제 충전(order: 참조) 줄이 차례로 쌓인 모습

결제위젯 오류 찾기

결제위젯이 뜨지 않거나 오류가 나면 브라우저 개발자 도구 콘솔의 오류 문구를 그대로 복사해 에이전트에게 주고, 토스페이먼츠 MCP로 원인을 찾게 합니다. 오류 코드와 SDK 버전

(v2)을 정확히 적을수록 맞는 문서를 찾습니다. 클라이언트 키가 `test_gck` 로 시작하는 결제위젯용 키인지도 확인합니다. 개발자센터에 로그인한 채 문서를 보면 일부 코드 예제의 키가 내 체험 상점의 API 개별 연동 키(`test_ck` 로 시작)로 바뀌는데, 이 키로는 결제위젯을 띄울 수 없습니다.

결과 해석: 돈이 움직인 기록이 세 곳에 남는다

결제 한 번이 끝나면 같은 사실이 세 곳에 남습니다. 토스페이먼츠의 결제 기록, 우리 `orders` 의 완료 주문, `credit_ledger` 의 + 줄입니다. 오늘 쓴 문서용 테스트 키로는 토스페이먼츠 쪽 기록을 개발자센터 화면에서 볼 수 없어 승인 응답을 저장한 approved_at 으로 대신했고, 내 키를 받은 뒤에는 개발자센터 결제 내역이나 결제 조회 API로 직접 볼 수 있습니다. 세 곳의 금액과 주문 번호가 서로 맞아야 정상이고, 하나라도 다르면 어디서 끊겼는지 찾을 단서가 됩니다. 예를 들어 토스페이먼츠에는 완료인데 우리 주문은 대기라면 승인 뒤 기록하다 실패한 것입니다. 실제 서비스는 이런 어긋남을 매일 맞춰 보는 대사(reconciliation) 작업을 따로 둡니다.

배포한 서비스로 옮기기

내 컴퓨터에서 확인했으면 staging에도 붙여 봅니다. 시크릿 키는 Secret Manager에 `toss-secret-staging` 같은 이름으로 넣고 런타임 서비스 계정에 그 시크릿 하나만 읽게 한 뒤, 서비스에 시크릿 이름으로 연결합니다. 클라이언트 키는 일반 환경변수로 넣습니다. successUrl과 failUrl은 staging 주소로 바뀌므로, 화면 코드가 지금 열린 주소를 기준으로 만들게 되어 있는지 확인합니다. 테스트 키를 쓰는 동안에는 staging에서도 실제 돈이 빠지지 않습니다.

코딩 에이전트에게 맡길 때: staging에 결제 키 연결

```
Secret Manager 시크릿 toss-secret-staging 은 내가 콘솔에서 만들었어. 값은 읽거나 출력하지 마.
friend-agent-runtime 서비스 계정에 이 시크릿 하나만 읽기 권한을 주고,
friend-agent-staging 서비스에 TOSS_SECRET_KEY 는 이 시크릿으로, TOSS_CLIENT_KEY 는 일반 환경변수로 연결하는 명령을 보여 줘.
클라이언트 키 값은 내가 직접 붙여 넣을게. 다른 설정은 바꾸지 마.
```

Chapter07

같은 요청이 두 번 오는 이유

사용자가 성공 페이지를 새로고침하거나, 네트워크가 느려 브라우저가 요청을 다시 보내거나, 버튼을 두 번 누르면 같은 승인 요청이 서버에 두 번 옵니다. 두 번째 요청도 금액이 맞고 주문도 있으므로, 아무 장치가 없으면 크레딧이 두 번 들어갑니다. 같은 요청을 여러 번 받아도 결과가 한 번 받은 것과 같게 만드는 성질을 멱등성(idempotency)이라고 합니다.

같은 일이 우리 서버와 토스페이먼츠 사이에서도 일어납니다. 서버가 승인 API를 불렀는데 응답이 오기 전에 연결이 끊기면, 서버는 승인이 됐는지 모른 채 같은 요청을 다시 보내야 합니다. 토스페이먼츠는 이때를 위해 멱등키를 받습니다. 승인 요청의 헤더에 `Idempotency-Key`로 고유한 값을 넣어 보내면, 토스페이먼츠는 같은 키, 같은 API 키, 같은 주소로 온 두 번째 요청을 다시 처리하지 않고 첫 응답을 그대로 돌려줍니다. 멱등키는 300자 이하이고 처음 쓴 날부터 15일 동안 유효합니다(2026년 11월 토스페이먼츠 문서 기준).

멱등키는 승인 API를 부르는 데 꼭 필요한 값이 아니라서, 섹션 6처럼 짧게 요청하면 에이전트가 넣지 않고 지나가기 쉽습니다. 그래서 이 섹션은 지금 만든 승인 처리가 멱등키를 보내는지부터 확인합니다.

중복을 막는 세 겹 장치

장치	막는 것
주문 상태 확인	이미 완료된 주문이면 승인 API를 다시 부르지 않음
원장 참조 유일	같은 `order:주문번호`로 + 줄을 두 번 쌓으려 하면 데이터베이스가 거절
토스페이먼츠 Idempotency-Key	같은 키로 승인 API를 다시 부르면 토스페이먼츠가 다시 처리하지 않고 첫 응답을 돌려줌

한 겹만으로도 대부분 막히지만, 두 요청이 거의 동시에 와서 둘 다 "아직 대기"를 읽는 경우처럼 한 겹이 뚫리는 상황이 있어 겹쳐 둡니다. 주문 상태 확인과 원장 참조는 우리 서버 안의 장치이고, Idempotency-Key는 서버가 토스페이먼츠로 요청을 다시 보낼 때 승인이 두 번 일어나지 않게 하는 장치입니다.

세 겹 장치가 각각 어느 상황을 막는지는 아래에서 장치를 하나씩 꺼 보며 확인합니다. 상황을 고르고 재생을 누르면 요청이 움직이고, 끝나면 실제로 빠진 돈, 원장에 쌓인 크레딧, 주문 상태가 나옵니다. 세 상황 모두에서 장치를 하나씩 끄고 다시 재생해, 결과가 나빠지는 조합을 찾아봅니다.

코딩 에이전트에게 맡길 때: 멱등키 보내기

```
결제 승인 API를 부를 때 Idempotency-Key 헤더를 보내고 있는지 확인해 줘. 근거는 토스페이먼츠 MCP로 멱등키 문서를 찾아 써 줘.
보내고 있지 않다면 같은 주문의 승인 요청에는 항상 같은 값이 가도록 추가해 줘. 값을 무엇으로 만들지는 나한테 물어보고 정해.
```

```
승인 API를 부를 때마다 orderId 와 Idempotency-Key 값을 서버 로그에 남겨 줘. 시크릿 키와 paymentKe
y 전체는 남기지 마.
```

직접 확인할 곳: 멱등키 확인

1. 에이전트가 알려 준 승인 코드의 파일과 줄을 열어, 승인 API 요청 헤더에 `Idempotency-Key` 가 들어가는지 봅니다.
2. 테스트 결제를 한 번 끝까지 진행하고, 서버 터미널 로그에 orderId 와 Idempotency-Key 값이 찍혔는지, 그 값이 아래에서 정한 방식대로 만들어졌는지 봅니다.

Idempotency-Key 값 정하기

요청을 보낼 때마다 새 무작위 값을 만들면, 다시 보낸 요청도 토스페이먼츠에는 새 요청으로 보여 멱등키가 아무것도 막지 못합니다. 같은 주문의 승인 요청이면 몇 번을 다시 보내도 같은 값이 가야 하므로, 주문을 만들 때 정해 둔 값을 씁니다. 섹션 6에서 주문 번호를 추측할 수 없는 무작위 값으로 만들었다면 주문 번호를 그대로 써도 되고, 따로 무작위 값을 만들어 주문에 저장해 두어도 됩니다.

첫 요청이 아직 처리 중일 때 같은 키로 다시 보내면 토스페이먼츠는 409(`IDEMPOTENT_REQUEST_PROCESSING`)를 돌려주므로, 이 응답이면 잠시 뒤 같은 키로 다시 물어 결과를 받습니다. 오류가 났다고 키를 바꿔 다시 보내면 승인이 두 번 일어날 수 있어 키는 바꾸지 않습니다. 승인 API가 성공했는데 우리 데이터베이스에 기록하다 오류가 나면 돈은 빠졌는데 크레딧이 없는 상태가 되는데, 이때도 같은 키로 다시 승인을 요청하면 첫 응답을 다시 받아 기록을 이어 갈 수 있고 원장 참조가 유일해서 크레딧이 두 번 쌓이지 않습니다.

직접 확인하기

멱등키를 넣었으면 브라우저에서 두 가지 상황을 직접 만들어 봅니다. 서버를 실행한 터미널을 옆에 띄워 두고, 승인 API를 부를 때 남긴 로그를 함께 봅니다.

상황	해 보는 방법	기대하는 결과
같은 승인 요청 두 번	결제를 끝낸 뒤 충전 완료 안내가 뜬 성공 페이지를 두세 번 새로고침	잔액이 처음 충전한 만큼에서 더 늘지 않고, `credit_ledger` 의 + 줄이 하나뿐임. 서버 로그에 승인 호출이 여러 번 찍혔다면 Idempotency-Key 값이 모두 같음
결제 취소	결제위젯에서 결제를 진행하다 취소	실패 페이지에 오류 문구, 주문은 실패, 원장 그대로

같은 승인 요청을 새로고침했을 때 화면에는 충전 완료 안내가 그대로 다시 뜰 수 있습니다. 서버가 이미 처리한 주문에 첫 처리 결과를 그대로 돌려주게 만들었다면 정상이고, 토스페이

먼츠의 멱등키가 동작하는 방식과도 같습니다. 그래서 이 상황은 화면의 문구가 아니라 잔액과 `credit_ledger`의 줄 수로 판단합니다. 안내가 다시 뜨면서 잔액도 함께 늘었다면 중복 지급이 일어난 것입니다.

승인 API 자체가 실패하는 경우(카드사 거절 등)는 테스트 환경에서 요청 헤더로 재현할 수 있습니다. 재현해 보고 싶다면 에이전트에게 토스페이먼츠 MCP로 테스트 환경 에러 재현 방법을 찾아 근거로 쓰게 합니다.

테스트로 남기기

금액 조작과 다른 사용자의 주문은 브라우저에서 직접 만들기 어렵습니다. 성공 페이지는 열리자마자 주소의 값으로 승인 요청을 보내므로, 주소의 amount를 고치거나 다른 창으로 옮기기 전에 원래 금액으로 승인이 끝나 버립니다. 이 두 상황은 테스트로 확인하고, 직접 확인한 두 상황도 코드를 고칠 때마다 다시 확인해야 하므로 함께 테스트로 남깁니다. 테스트로 만들어 두면 `pytest` 한 번으로 매번 확인할 수 있습니다. 테스트에서는 실제 토스페이먼츠를 부르지 않고 승인 API 호출 부분을 가짜로 바꿔 끼웁니다. 가짜 LLM으로 대화를 시험하는 것과 같은 방식입니다.

코딩 에이전트에게 맡길 때: 결제 테스트 만들기

```
결제 승인 테스트를 tests/ 에 추가해 줘. 상황은 네 가지야: 같은 승인 요청 두 번, 금액을 바꾼 요청, 다른 사용자의 주문, 승인 실패.
금액을 바꾼 요청과 다른 사용자의 주문은 가짜 승인 API가 한 번도 불리지 않았는지도 확인해 줘.
실제 토스페이먼츠는 부르지 말고 승인 API 호출을 가짜로 바꿔 끼우고, 가짜 응답 모양은 토스페이먼츠 MCP로 결제 승인 문서를 찾아 실제와 같게 만들어 줘.
필요한 추가 스펙은 나한테 물어봐서 확정된 이후에 만들고, 기존 테스트는 바꾸지 마. 테스트마다 무엇을 확인하는지 한 줄씩 설명해 줘.
```

직접 실행할 명령: 결제 테스트 실행

```
# 저장소 루트, 가상환경을 켠 터미널에서
pytest -q
```

결과 해석: 테스트가 지키는 약속

결제 테스트가 모두 통과하면, 새로고침이나 주소 조작이 있어도 크레딧은 실제로 승인된 금액만큼 한 번만 들어간다는 것을 매번 자동으로 확인할 수 있습니다. 테스트가 하나라도 실패하면 그 상황이 실제로 일어났을 때 돈과 크레딧이 어긋난다는 뜻이므로, 배포 전에 반드시 고칩니다. CI가 있다면 이 테스트도 PR마다 함께 돕니다.

주관식 퀴즈

Q. [이해 점검] 같은 승인 요청이 거의 동시에 두 번 들어와서, 두 요청 모두 주문 상태를 "대기"로 읽었습니다. 이 경우 크레딧이 두 번 들어가지 않으려면 지금 구조의 어느 장치가 막아 주는지, 그리고 그 장치가 없었다면 어떤 일이 생겼을지 설명해 보세요.

Chapter08

오늘 배운 것

원가는 사용량에 비례한다. 메시지 한 건에는 LLM 호출이 여러 번 들어가고, 원가는 그 호출들의 토큰 비용을 더한 값입니다. Langfuse 기록으로 평균과 비싼 쪽 원가, 대화 길이에 따른 차이를 읽었고, 메시지 한 건은 몇 원이 안 되어도 많이 쓰는 사용자와 자동화 악용 계정은 보통 사용자의 열 배에서 백 배 넘는 원가를 만든다는 것을 계산했습니다.

가격은 원가, 수수료, 고정비, 무료 한도로 정한다. 원가가 작은 서비스에서 원가는 가격의 바닥만 정하고, 한 달 손익을 가장 크게 바꾸는 것은 무료 한도와 전환율이었습니다. 무료 한도는 전환율을 재기 전까지 작게 잡습니다. 큰 모델로 바꾸면 원가가 단가 차이만큼 늘고, 원가를 줄이는 방법은 모두 품질과 맞바꾸므로, 어느 쪽이든 평가로 확인한 뒤에 씁니다.

크레딧은 원장으로, 차감은 서버에서. 잔액을 덮어쓰지 않고 거래를 쌓았고, 참조를 유일하게 두어 같은 일이 두 번 기록되지 않게 했습니다.

결제는 서버가 승인해야 끝난다. 주문을 먼저 저장하고, successUrl의 금액을 저장한 금액과 비교하고, 시크릿 키로 승인한 뒤에만 크레딧을 지급했습니다. 승인 요청에는 같은 주문이면 같은 값이 가는 멱등키를 넣었고, 같은 요청 두 번, 금액 조작, 다른 사용자의 주문, 승인 실패를 테스트로 막았습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 하루 300개를 보내는 사용자 때문에 생기는 문제를 적었습니다. 그 답을 다시 읽고, 오늘 만든 원가표, 크레딧 한도, 결제 흐름이 그 문제를 어디까지 막는지, 아직 막지 못하는 것은 무엇인지 적어 보세요.